

IoT Challenge #3 – Part 1

Node-RED Challenge Report

Mobin Khatib, Ali Edareh Heidarabdi
11114435, 11036028

May 2026

1 Introduction

This report presents the implementation of Part 1 of IoT Challenge #3 using Node-RED. The objective of the challenge was to design a Node-RED flow able to generate random ID messages, publish them to a local MQTT broker, subscribe to the same MQTT topic, process the corresponding rows of the `challenge3.csv` file, extract ZigBee information, generate the required CSV files, display RMS Current and RMS Voltage on dashboard charts, and send selected outgoing cost values to ThingSpeak.

The local Mosquitto broker was configured on:

```
localhost:1884
```

The MQTT topic used for the generated ID messages was:

```
challenge3/id_generator
```

The flow was programmed to stop after receiving exactly 200 ID messages from the MQTT subscription. Ignored packets were also counted toward this 200-message limit, as required by the challenge specification.

2 Complete Node-RED Flow

Figure 1 shows the complete Node-RED flow implemented for the challenge. The flow is composed of four main levels:

Level 1: Initialization and reset of CSV files.

Level 2: Periodic generation of random ID messages and publication to MQTT.

Level 3: Subscription to the generated ID topic and computation of the packet number N.

Level 4: Processing of `challenge3.csv` rows and generation of the required outputs.

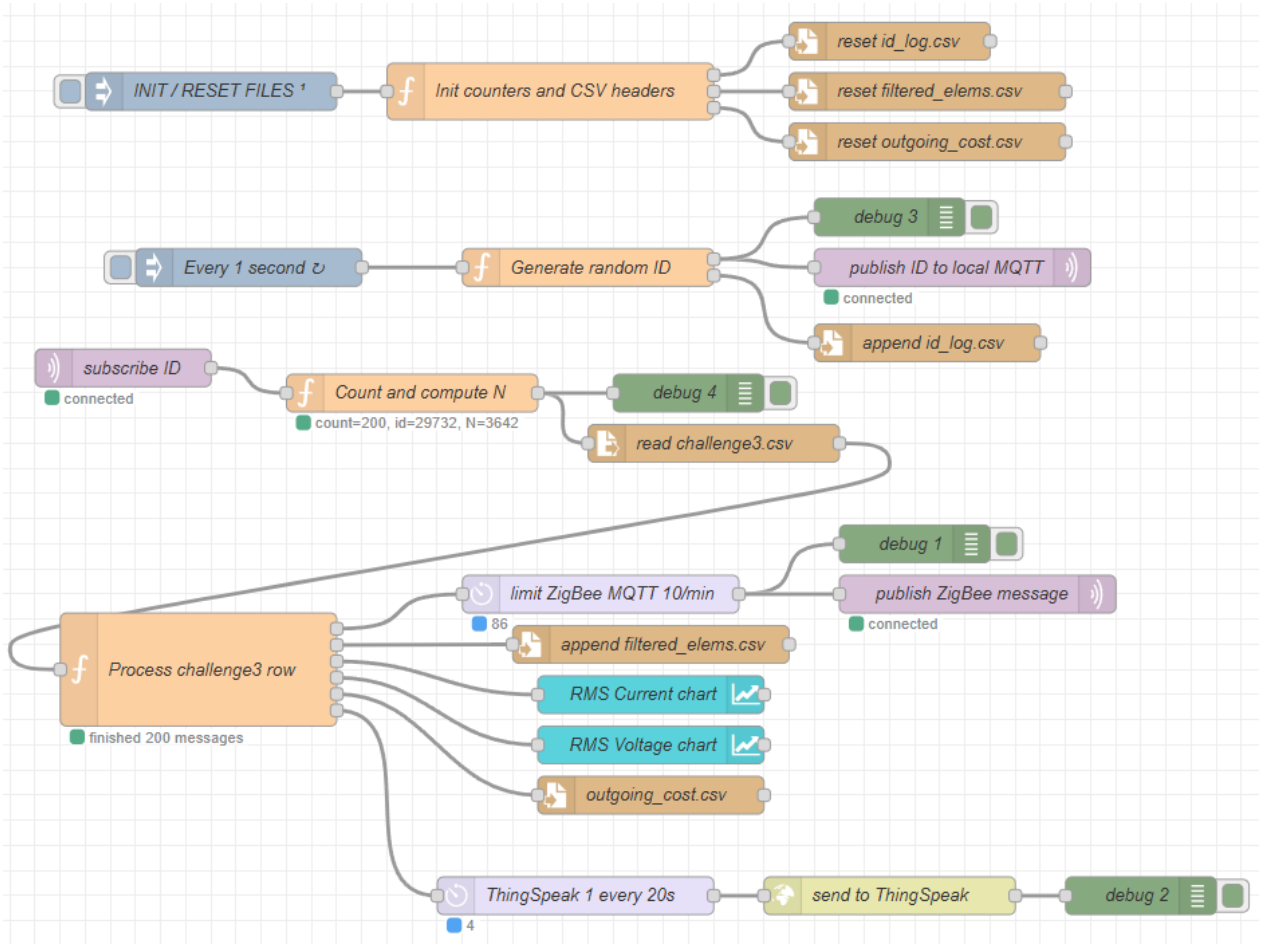


Figure 1: Complete Node-RED flow used for Challenge 3.

3 Level 1: Initialization and CSV File Reset

The initialization part of the flow is executed automatically when the flow is deployed. It prepares all counters and resets the output CSV files.

The main nodes in this level are:

- **INIT / RESET FILES:** inject node executed once at deployment.
- **Init counters and CSV headers:** function node used to initialize flow variables and generate CSV headers.
- **reset id_log.csv:** file node used to overwrite and initialize `id_log.csv`.
- **reset filtered_elems.csv:** file node used to overwrite and initialize `filtered_elems.csv`.
- **reset outgoing_cost.csv:** file node used to overwrite and initialize `outgoing_cost.csv`.

The initialized flow variables are:

- **stop:** Boolean flag used to stop the execution after 200 subscribed messages.
- **idRowNo:** incremental row number for `id_log.csv`.
- **receivedCount:** number of messages received from the subscription.
- **filteredRowNo:** incremental row number for `filtered_elems.csv`.

- **outgoingCosts**: object used to store and update outgoing costs from Link Status packets.

The CSV headers generated by this part are:

```
id_log.csv:
No.,ID,TIMESTAMP

filtered_elems.csv:
No.,Timestamp,Sequence Number,Attribute,Status,Data Type,Data Value

outgoing_cost.csv:
No.,Source,Destination,Cost
```

4 Level 2: Random ID Generation Branch

The first separated branch of the flow is shown in Figure 2. This branch periodically generates random IDs, publishes them through MQTT, and stores them in `id_log.csv`.

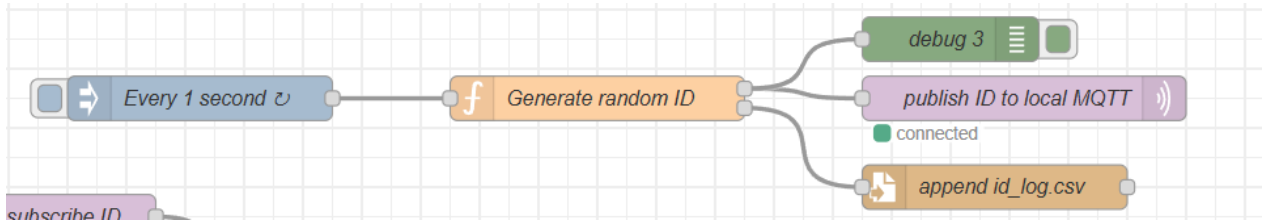


Figure 2: Random ID generation, MQTT publication, and ID logging branch.

This branch contains the following nodes:

- **Every 1 second**: inject node configured to trigger the flow every second.
- **Generate random ID**: function node that creates a random ID and timestamp.
- **debug 3**: debug node used to inspect the generated message.
- **publish ID to local MQTT**: MQTT output node publishing the generated message.
- **append id_log.csv**: file node appending each generated ID to the CSV file.

The `Generate random ID` node creates an ID between 0 and 30000:

```
let id = Math.floor(Math.random() * 30001);
```

The timestamp is generated as a UNIX timestamp:

```
let timestamp = Math.floor(Date.now() / 1000);
```

Each MQTT message has the following JSON format:

```
{
  "id": 7123,
  "timestamp": 1710930219
}
```

The same information is saved in `id_log.csv` with the format:

```
No.,ID,TIMESTAMP
```

The branch also checks the `stop` flag. When the required limit is reached, no additional ID messages are generated.

5 Level 3: Subscription and Computation of N

The second separated branch is shown in Figure 3. This part subscribes to the generated ID messages, counts the received messages, computes N, and reads the challenge CSV file.

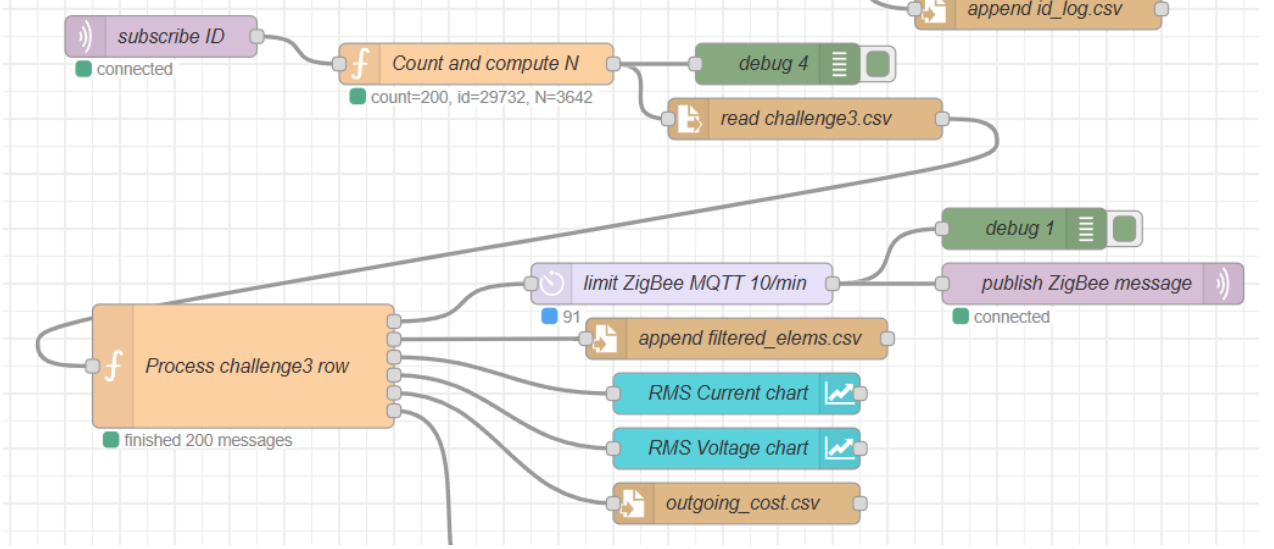


Figure 3: MQTT subscription, received-message counting, computation of N, and reading of challenge3.csv.

This branch contains:

- **subscribe ID**: MQTT input node subscribed to challenge3/id_generator.
- **Count and compute N**: function node that extracts the received ID and computes N.
- **debug 4**: debug node used to monitor the received ID, counter, and computed N.
- **read challenge3.csv**: file input node used to read the complete CSV file.

For each received message, the ID is extracted from the MQTT payload. Then N is computed as:

$$N = ID \bmod 5218$$

For example, if the received ID is 7123:

$$N = 7123 \bmod 5218 = 1905$$

The value N is then used to select the row of challenge3.csv whose Packet Number is equal to N. The node status confirms the progress of the execution. In the final run, the status showed:

count=200, id=29732, N=3642

This confirms that exactly 200 subscribed ID messages were processed.

6 Level 4: Processing and Output Generation Branch

The third separated branch is shown in Figure 4. This is the main processing part of the flow. It receives the content of challenge3.csv, finds the row corresponding to the computed packet number N, and produces the required outputs.

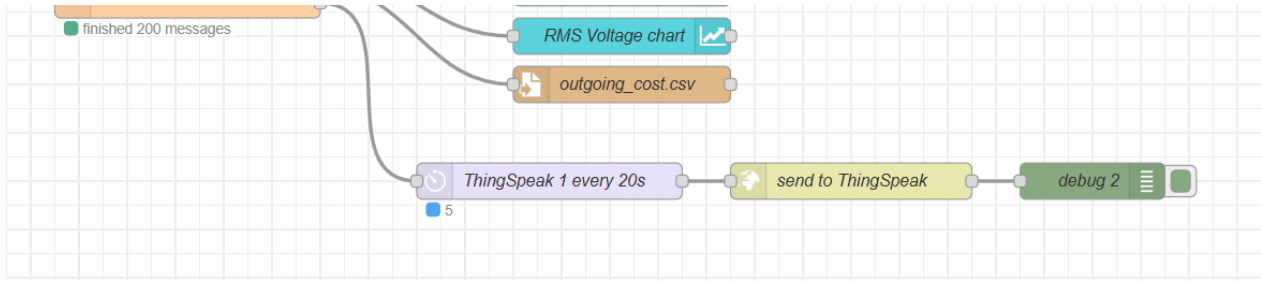


Figure 4: Main processing branch and output generation nodes.

The core node of this branch is:

Process challenge3 row

This function node has six outputs:

1. MQTT publication of selected ZigBee messages.
2. Appending selected elements to `filtered_elems.csv`.
3. Sending RMS Current values to the Node-RED chart.
4. Sending RMS Voltage values to the Node-RED chart.
5. Writing final outgoing costs to `outgoing_cost.csv`.
6. Sending selected outgoing cost values to ThingSpeak.

6.1 CSV Parsing and Packet Selection

The function node first parses the full `challenge3.csv` file. Then it searches for the row where:

$$\text{Packet Number} = N$$

If no row is found, the message is ignored. If the row is found, the `Command String` and packet type are analyzed.

6.2 Case 1: ZBEE_ZCL Message

If the selected row contains a ZBEE_ZCL layer inside the `Command String`, the flow generates an MQTT message.

The MQTT topic is based on:

Device Name ZigBee Source

The payload is generated in JSON format:

```

{
  "timestamp": CURRENT_TIMESTAMP,
  "id": SUB_ID,
  "wpan.src": "SRC_ADDR",
  "wpan.dst": "DST_ADDR",
  "zbee.src": "ZB_SRC_ADDR",
  "zbee.dst": "ZB_DST_ADDR",
  "topic": "ZigBee/ZB_SRC_DEV_NAME",
  "payload": "CMD_STR"
}
  
```

The meaning of each field is:

- **timestamp**: current UNIX timestamp when the message is prepared.
- **id**: ID received from the MQTT subscription.
- **wpan.src**: source address from the CSV row.
- **wpan.dst**: destination address from the CSV row.
- **zbee.src**: ZigBee source address from the CSV row.
- **zbee.dst**: ZigBee destination address from the CSV row.
- **topic**: ZigBee topic using the ZigBee source device name.
- **payload**: original command string from the selected CSV row.

All addresses are converted to hexadecimal format. For example:

```
0x0000
0x0112
0x1cd8
```

The generated ZigBee MQTT message passes through the node:

```
limit ZigBee MQTT 10/min
```

This delay node is configured as a rate limiter, so the publication rate is limited to 10 messages per minute, as required.

6.3 Filtered Elements Extraction

For ZBEE.ZCL packets, the flow also checks whether the **Command String** contains one or more of these attributes:

- Active Power
- RMS Current
- RMS Voltage

If these attributes are found, the function extracts:

- Timestamp
- Sequence Number
- Attribute
- Status
- Data Type
- Data Value

The extraction is done by matching the elements by their position in the lists. Therefore:

- **Attribute[0]** is matched with **Status[0]**, **Data Type[0]**, and the first corresponding value.
- **Attribute[1]** is matched with **Status[1]**, **Data Type[1]**, and the second corresponding value.
- **Attribute[2]** is matched with **Status[2]**, **Data Type[2]**, and the third corresponding value.

The selected rows are appended to `filtered_elems.csv` using the required format:

No.	Timestamp	Sequence Number	Attribute	Status	Data Type	Data Value
-----	-----------	-----------------	-----------	--------	-----------	------------

An example of the generated structure is:

1	1778534716	8	Active Power	Success	16-Bit Signed Integer	0
2	1778534716	8	RMS Current	Success	16-Bit Unsigned Integer	0
3	1778534716	8	RMS Voltage	Success	16-Bit Unsigned Integer	221

6.4 RMS Current and RMS Voltage Dashboard Charts

The RMS Current and RMS Voltage values are also sent to two Node-RED dashboard chart nodes:

- RMS Current chart
- RMS Voltage chart

The RMS Current chart receives only the extracted RMS Current values. In the obtained execution, the RMS Current values were equal to 0 for the displayed samples.

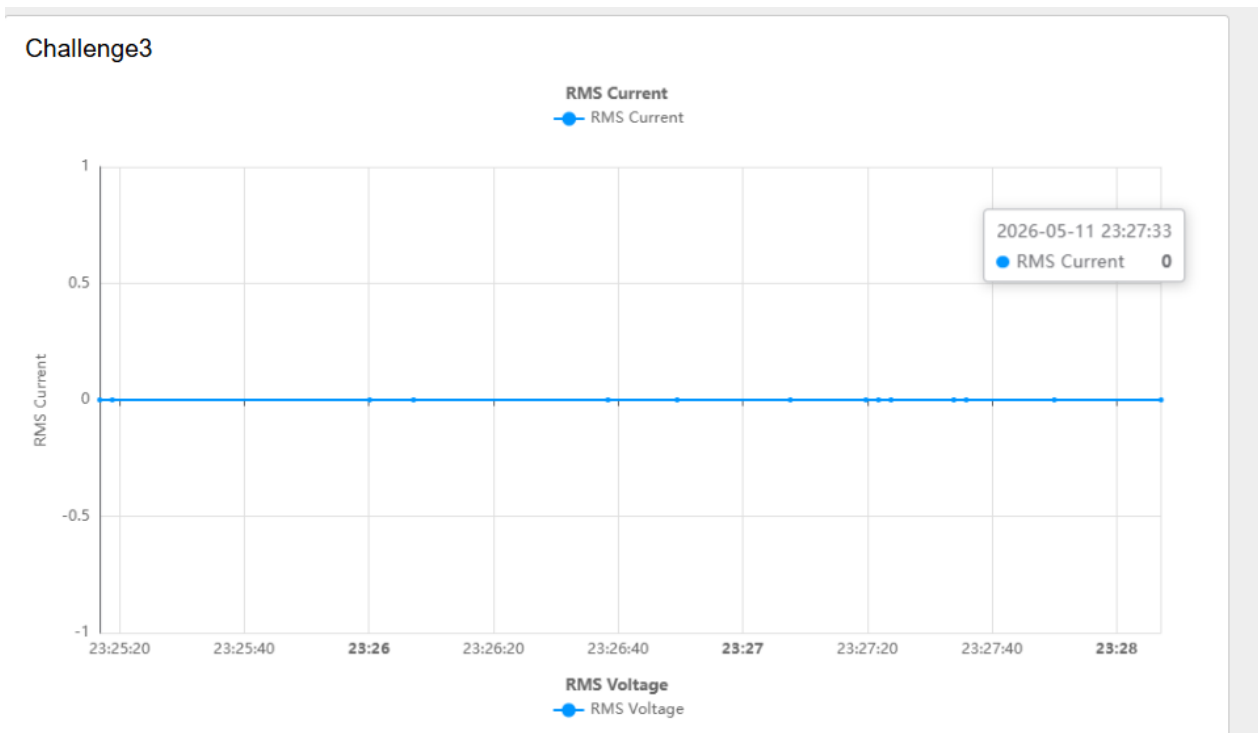


Figure 5: Node-RED dashboard chart for RMS Current.

The RMS Voltage chart receives the extracted RMS Voltage values. In the obtained execution, the values were approximately between 219 and 223.

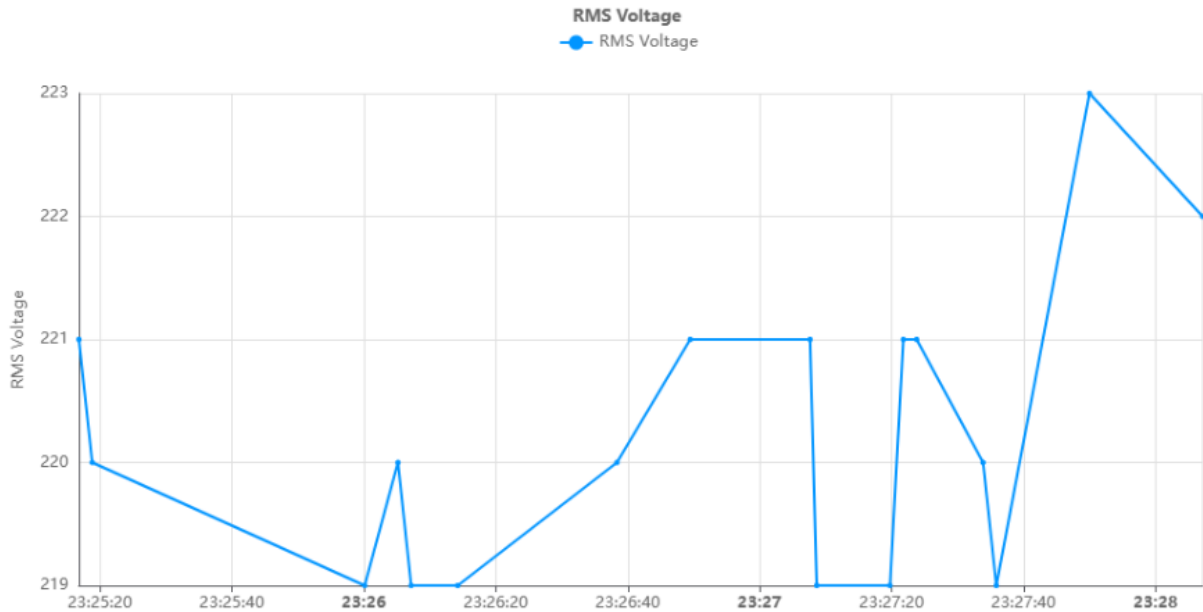


Figure 6: Node-RED dashboard chart for RMS Voltage.

6.5 Case 2: Link Status Message

If the selected row does not contain a ZBEE_ZCL layer but contains a Link Status message, the flow extracts outgoing cost information.

The source address is taken from the ZigBee source address field. For each link inside the Link Status command string, the destination address and outgoing cost are extracted.

The values are stored in the internal object:

```
outgoingCosts
```

If another Link Status packet is later received from the same source and destination, the previous cost is updated with the newer value. This satisfies the requirement that later Link Status packets must update the stored cost.

At the end of execution, after exactly 200 received ID messages, the final values are written to:

```
outgoing_cost.csv
```

The format is:

```
No.,Source,Destination,Cost
```

An example of the produced structure is:

```
1,0x0000,0x0112,7
2,0x0000,0x1cd8,1
3,0x0000,0x1e15,3
4,0x0000,0x3d95,3
5,0x0000,0x4e11,3
```

The source and destination addresses are sorted by treating them as hexadecimal numbers.

6.6 Case 3: Other Packets

If the selected packet is neither a ZBEE_ZCL message nor a Link Status message, the packet is ignored. However, it is still counted as one received ID message. This is important because the flow must stop after exactly 200 subscribed ID messages, regardless of whether the packet was processed or ignored.

7 Stopping Condition

The flow is controlled by the variable:

```
receivedCount
```

Each message received by the MQTT subscription increments this counter. When the counter reaches 200, the variable `stop` is set to `true`. After this point, the flow does not process more ID messages.

The status of the main processing node showed:

```
finished 200 messages
```

This confirms that the execution stopped at the correct limit.

8 ThingSpeak Transmission

After the 200-message execution is completed, the function finds the smallest source address in `outgoing_cost.csv`. The addresses are treated as hexadecimal numbers.

For the smallest source address, all destination addresses are sorted in ascending hexadecimal order. Then the corresponding outgoing cost values are sent to ThingSpeak using the HTTP API.

Each value is sent to:

```
field1
```

The messages are passed through the delay node:

```
ThingSpeak 1 every 20s
```

This node guarantees the required rate limit of one ThingSpeak update every 20 seconds. The public ThingSpeak channel used for this challenge is:

<https://thingspeak.mathworks.com/channels/3378724/>

Figure 7 shows the ThingSpeak plot obtained from the transmitted outgoing cost values.

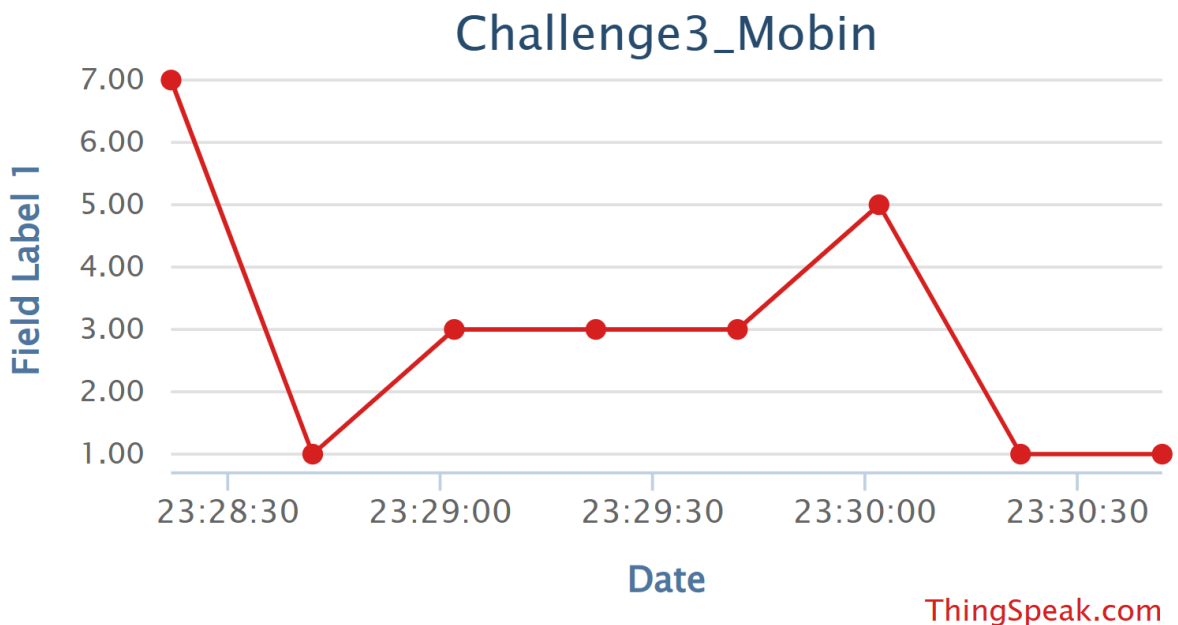


Figure 7: ThingSpeak field 1 plot showing the transmitted outgoing cost values.

9 Produced Output Files

The following files were produced by the Node-RED flow and included in the delivery:

- `nodered.txt`
- `id_log.csv`
- `filtered_elems.csv`
- `outgoing_cost.csv`

9.1 `id_log.csv`

The file `id_log.csv` contains the generated random IDs and their timestamps. The format is:

No.,ID,TIMESTAMP

This file contains 200 generated ID rows plus the header.

9.2 `filtered_elems.csv`

The file `filtered_elems.csv` contains the extracted Active Power, RMS Current, and RMS Voltage elements from the selected ZBEE_ZCL packets.

The format is:

No.,Timestamp,Sequence Number,Attribute,Status,Data Type,Data Value

9.3 `outgoing_cost.csv`

The file `outgoing_cost.csv` contains the final outgoing cost values extracted from Link Status packets.

The format is:

No.,Source,Destination,Cost

Both source and destination addresses are reported in hexadecimal format.

10 Conclusion

The implemented Node-RED flow satisfies the requirements of Part 1 of IoT Challenge #3. The flow generates random IDs every second, publishes them to a local MQTT broker, subscribes to the same topic, computes the packet number N, processes the corresponding row of `challenge3.csv`, extracts ZigBee information, creates the required CSV files, plots RMS Current and RMS Voltage on the Node-RED dashboard, and sends selected outgoing costs to ThingSpeak.

The execution correctly stops after exactly 200 received ID messages. The produced output files follow the required CSV formats, and the ThingSpeak transmission is rate-limited to one message every 20 seconds.